

五子棋图形界面游戏实验报告

1. 实验目的

- 使用 C++ 面向对象方法，完成一个完整项目的架构设计与编码。
- 学习 Qt 框架的 QWidget、QPainter 和信号槽机制 (Signal & Slot)，实现图形用户界面。
- 在数据层、逻辑层、视图层之间做分层设计，降低耦合。

2. 实验环境

项目	说明
操作系统	Linux
编程语言	C++17
GUI 框架	Qt 6.5 (Widgets)
构建工具	CMake 3.19+
编译器	GCC
版本控制	Git

3. 系统设计与架构

3.1. 总体设计思路

项目采用自底向上、分层渐进的方式开发：从只依赖标准库的 C++ 控制台的棋盘点阵实现开始，逐步扩展到 Qt 图形界面。

系统分为四层：

- 第 4 层：MainWindow (UI 串联层)**：负责按钮事件、状态栏更新、胜负弹窗。通过 connect 连接各组件的信号槽，系统中唯一知道所有组件存在的类。
- 第 3 层：Game (控制层)**：游戏流程控制：落子、判胜负、切换回合、悔棋。持有 Board 实例，通过信号通知外层。只关心规则逻辑，不涉及 UI 绘制。
- 第 2 层：Board (数据 + 规则层)**：棋盘数据存储（一维数组存储 15×15 棋子状态）及胜负判定。纯 C++ 类，零 Qt 依赖，可在控制台中独立测试。
- 第 1 层：ChessBoardWidget (视图层)**：负责绘制棋盘网格、星位、棋子，处理鼠标点击交互。持有 `const Board*` 指针从 Board 读取数据来画，不自行存储棋子副本。用户点击后发射 `cellClicked` 信号。

层间依赖关系：

- 上层持有下层对象的引用或指针，通过调用下层方法驱动系统。
- 下层通过 Qt 信号向上层通知状态变化，但下层代码中不包含任何对上层的引用。

因此 Board 可以独立测试，Game 可以在没有 UI 的情况下运行，视图层可以替换而不修改逻辑。

3.2. 信号与数据流向

一次完整的落子操作经过以下数据流：

1. 用户点击棋盘，ChessBoardWidget::mousePressEvent 被触发。
2. 像素坐标经 pixelToCell 转换为棋盘行列 (row, col)。
3. ChessBoardWidget 发射 cellClicked(row, col) 信号。
4. MainWindow 收到信号，调用 Game::placePiece(row, col)。
5. Game 内部调用 Board::placeAt(col, row, chess) 更新数据。
6. Game 调用 Board::hasFiveAt() 判定胜负。
7. Game 根据结果发射 piecePlaced、currentPlayerChanged 或 gameOver 信号。
8. MainWindow 收到信号后：
 - 调用 ChessBoardWidget::update() 触发重绘。
 - 更新状态栏文字（“当前：黑方落子”）。
 - 若游戏结束，弹出 QMessageBox 公告胜负。

3.3. 设计决策

单一数据源。 整个程序只有一份棋盘数据。ChessBoardWidget 不维护自己的棋子数组，而是通过 const Board* 指针读取 Game 内部的 Board 对象。

单向依赖。 Board 不知道 Game 的存在，Game 不知道 MainWindow 的存在。上层调用下层的方法，下层通过信号通知上层。更换 UI 框架时只需重写视图层和控制层。

4. 详细实现过程

开发从底层到顶层顺序进行，每一步可以独立测试。

4.1. Board 类 —— 棋盘数据与规则（只依赖 C++ 标准库，控制台测试）

最早实现的部分。Board 类完全独立于 Qt，只依赖 C++ 标准库。

数据结构选择： 使用 std::vector<Chess> 一维数组存储 $15 \times 15 = 225$ 个位置。通过 $\text{index}(x, y) = y * \text{width_} + x$ 将二维坐标映射到一维索引。相比二维 vector<vector<>>，一维数组内存连续，访问更快。

棋子枚举：

```
enum class Chess { kEmpty, kBlack, kWhite };
```

核心接口：

- placeAt(x, y, chess): 在 (x, y) 放置棋子，返回是否成功。包含越界检查和占位检查。
- at(x, y): 查询某位置的棋子类型。
- hasFiveAt(x, y, chess): 以 (x, y) 为中心点，在四个方向（横、竖、正斜、反斜）上向正反方向各延伸 4 步，若任一方总计数 ≥ 5 则该方获胜。
- hasFiveInBoard(chess): 扫描棋盘，判断指定棋子是否存在任意方向的五连。
- clear(): 使用 std::fill 清空棋盘。
- forceClear(x, y): 强制清除指定位置（悔棋专用，不检查占位）。
- printBoardToTerminal(): 控制台打印棋盘，空位为 *，黑棋为 x，白棋为 o。

placeAt 实现（含越界和占位检查）：

```
bool Board::placeAt(int x, int y, Chess chess) {
    if (!inBound(x, y)) {
        return false;
    }
    if (at(x, y) != Chess::kEmpty) {
        return false;
    }
}
```

```

    pieces_[index(x, y)] = chess;
    return true;
}

```

hasFiveAt 实现（四方向双向扫描）：

```

/* 以指定位置为中心，判断是否出现五连 */
bool Board::hasFiveAt(int x, int y, Chess chess) const {
    if (!inBound(x, y) || at(x, y) != chess) {
        return false;
    }

    const int kDirections[4][2] = {
        {1, 0}, // 横向
        {0, 1}, // 竖向
        {1, 1}, // 对角线方向
        {1, -1}, // 反对角线方向
    };

    // 判断该方向上是否有五个棋子相连
    for (const auto kDirection : kDirections) {
        int count = 1;

        for (int step = 1; step <= 4; ++step) {
            int next_x = x + kDirection[0] * step;
            int next_y = y + kDirection[1] * step;
            if (!inBound(next_x, next_y) || at(next_x, next_y) != chess) {
                break;
            }
            ++count;
        }
        for (int step = 1; step <= 4; ++step) {
            int next_x = x - kDirection[0] * step;
            int next_y = y - kDirection[1] * step;
            if (!inBound(next_x, next_y) || at(next_x, next_y) != chess) {
                break;
            }
            ++count;
        }

        // 有大于等于五个棋子相连
        if (count >= 5) {
            return true;
        }
    }

    return false;
}

```

坐标映射与清空方法：

```

// 二维 → 一维索引
std::size_t Board::index(int x, int y) const { return y * width_ + x; }

// 清空全盘
void Board::clear() {
    std::fill(pieces_.begin(), pieces_.end(), Chess::kEmpty);
}

// 悔棋专用：强制清除（覆盖已有棋子）
void Board::forceClear(int x, int y) {
    if (inBound(x, y)) pieces_[index(x, y)] = Chess::kEmpty;
}

```

坐标约定：Board 采用 (x, y) = (列, 行)，即 x 对应列号，y 对应行号。后续所有组件在调用 Board 接口时都需要做 (row, col) → (col, row) 的转换。

控制台测试： 在 main.cpp 中构造 Board，手动放置棋子，调用 printBoardToTerminal() 和 hasFiveAt() 验证五连判定逻辑。

```
// 测试代码示例
Board board(15, 15);
for (int i = 0; i < 5; ++i) board.placeAt(i, 5, Chess::kBlack);
board.printBoardToTerminal();
// 输出：对角方向有 5 个黑子，应判定为黑方获胜
```

4.2. 搭建 Qt 项目框架

CMakeLists.txt 配置：

```
find_package(Qt6 6.5 REQUIRED COMPONENTS Core Widgets)
target_link_libraries(five-in-a-row PRIVATE Qt::Core Qt::Widgets)
```

4.3. ChessBoardWidget —— 棋盘绘制与鼠标交互

这是项目中实现难度最高的部分。该控件继承自 QWidget，重写了 paintEvent 和 mousePressEvent。

4.3.1. 坐标系统设计

棋盘绘制需要处理像素坐标与棋盘逻辑坐标的相互转换。

以 600×600 像素的 Widget、15 路棋盘为例：

设格子尺寸为 S ，边距为 M ，棋盘路数为 $N = 15$ 。

总宽度满足： $W = 2M + (N - 1) * S$ ，取 $M = \frac{S}{2}$ 。

代入得： $W = S + (N - 1) * S = N * S$ ，故 $S = \frac{W}{N}$ 。

具体数值： $S = \frac{600}{15} = 40$ ， $M = \frac{40}{2} = 20$ 。

两种转换方式：

- 棋盘 $\rightarrow$ 像素： $px = margin + col * cellSize$ 、 $py = margin + row * cellSize$ （精确映射，用于绘制棋子）。
- 像素 $\rightarrow$ 棋盘： $col = \frac{px}{cellSize}$ 、 $row = \frac{py}{cellSize}$ （整数除法取商，利用格子均匀分布推算交叉点行列），无效点击返回 $(-1, -1)$ 。

```
int ChessBoardWidget::cellSize() const { return width() / board_size_; }
int ChessBoardWidget::margin() const { return cellSize() / 2; }
```

// 棋盘行列 $\rightarrow$ 像素坐标

```
QPoint ChessBoardWidget::cellToPixel(int row, int col) const {
    return QPoint{margin() + col * cellSize(),
                  margin() + row * cellSize()};
}
```

// 像素坐标 $\rightarrow$ 棋盘行列（带越界检查）

```
QPoint ChessBoardWidget::pixelToCell(int px, int py) const {
    int col = -1, row = -1;
    if (inBound(px, py)) {
        col = px / cellSize();
        row = py / cellSize();
    }
    return QPoint{row, col}; // x = row, y = col
}
```

4.3.2. paintEvent —— 绘制流程

使用 QPainter 在 Widget 上绘图。绘制顺序为：背景 $\rightarrow$ 网格线 $\rightarrow$ 星位 $\rightarrow$ 棋子。

```
void ChessBoardWidget::paintEvent(QPaintEvent*) {
    QPainter painter(this);
    painter.setRenderHint(QPainter::Antialiasing); // 抗锯齿
```

```

int margin = margin();
int cell_size = cellSize();
int last = board_size_ - 1;

// 1. 背景 (木色)
painter.fillRect(rect(), QColor{"#DEB887"});

// 2. 15 条横线 + 15 条竖线
painter.setPen(QPen{Qt::black, 1});
for (int i = 0; i < board_size_; ++i) {
    painter.drawLine(margin, margin + i * cell_size,
                     margin + last * cell_size,
                     margin + i * cell_size); // 横线
    painter.drawLine(margin + i * cell_size, margin,
                     margin + i * cell_size,
                     margin + last * cell_size); // 竖线
}

// 3. 星位 (天元 + 四角, 共 9 个)
int stars[][2] = {{3, 3}, {3, 7}, {3, 11},
                  {7, 3}, {7, 7}, {7, 11},
                  {11, 3}, {11, 7}, {11, 11}};
painter.setBrush(Qt::black);
painter.setPen(Qt::NoPen);
for (auto& s : stars) {
    if (s[0] < board_size_ && s[1] < board_size_) {
        QPoint c = cellToPixel(s[0], s[1]);
        painter.drawEllipse(c, 3, 3);
    }
}

// 4. 棋子: 遍历 Board, 绘制实心黑圆 / 空心白圆
int radius = cell_size * 0.4;
if (board_) {
    for (int row = 0; row < board_size_; ++row) {
        for (int col = 0; col < board_size_; ++col) {
            Chess chess = board_->at(col, row);
            if (chess == Chess::kEmpty) continue;

            QPoint center = cellToPixel(row, col);
            painter.setPen(Qt::black);
            if (chess == Chess::kBlack) {
                painter.setBrush(Qt::black);
            } else {
                painter.setBrush(Qt::white);
            }
            painter.drawEllipse(center, radius, radius);
        }
    }
}
}

```

相关 Qt 概念:

- QPainter: 绘图引擎。setPen 控制线条, setBrush 控制填充。
- paintEvent: Qt 在需要重绘时自动调用, 开发者的代码只需描述“画什么”, 不需要控制“何时画”。
- update(): 通知 Qt 需要重绘, Qt 在下一帧调用 paintEvent。

4.3.3. mouseReleaseEvent —— 点击处理

```

void ChessBoardWidget::mouseReleaseEvent(QMouseEvent* event) {
    QPoint cell = pixelToCell(event->pos().x(), event->pos().y());
    if (cell.x() == -1) return; // 无效点击
    if (board_ && !board_->isEmpty(cell.y(), cell.x())) return; // 已有棋子

    emit cellClicked(cell.x(), cell.y()); // 发射信号给 MainWindow
}

```

使用 `mouseReleaseEvent`（鼠标释放）而非 `mousePressEvent` 是为了避免拖拽误触：用户按住鼠标移动再松开时不会触发落子。

ChessBoardWidget 头文件声明：

```
class ChessBoardWidget : public QWidget {
    Q_OBJECT
public:
    explicit ChessBoardWidget(QWidget* parent = nullptr);
    void set_board(const Board* board);
    void set_board_size(int size);

protected:
    void paintEvent(QPaintEvent* event) override;
    void mouseReleaseEvent(QMouseEvent* event) override;

signals:
    void cellClicked(int row, int col);

private:
    int board_size_ = 15;
    const Board* board_ = nullptr;

    int cellSize() const;
    int margin() const;
    bool inBound(int px, int py) const;
    QPoint pixelToCell(int px, int py) const;
    QPoint cellToPixel(int row, int col) const;
};
```

4.4. Game 类 —— 游戏控制逻辑

Game 继承自 `QObject`，管理游戏流程，通过 Qt 的信号槽机制与 UI 层通信。

头文件声明：

```
class Game : public QObject {
    Q_OBJECT
public:
    explicit Game(int board_size = 15, QObject* parent = nullptr);

    bool placePiece(int row, int col);
    bool undo();
    void reset();

    Chess current_player() const;
    const Board& board() const;
    bool game_over() const;

signals:
    void piecePlaced(int row, int col, Chess chess);
    void pieceRemoved(int row, int col);
    void gameOver(Chess winner);
    void currentPlayerChanged(Chess player);
    void gameReset();

private:
    Board board_;
    Chess current_player_ = Chess::kBlack;
    std::stack<QPoint> move_history_;
    bool game_over_ = false;

    void switchPlayer();
};
```

核心属性：

- `Board board_`：棋盘实例。

- Chess currentPlayer_: 当前玩家（黑先）。
- std::stack<QPoint> moveHistory_: 落子历史记录（用于悔棋），x 存 row, y 存 col。
- bool gameOver_: 游戏结束标志。

placePiece 流程与实现：

1. 检查游戏是否已结束。
2. 调用 board_.placeAt(col, row, currentPlayer_)（注意坐标转换：输入 (row, col)，传给 Board 时交换为 placeAt(col, row)）。
3. 压入历史记录。
4. 发射 piecePlaced 信号。
5. 调用 board_.hasFiveAt() 判断是否五连获胜。
6. 若获胜则发射 gameOver 信号，否则 switchPlayer() 切换玩家并发射 currentPlayerChanged 信号。

```
bool Game::placePiece(int row, int col) {
    if (game_over_) return false;

    if (!board_.placeAt(col, row, current_player_)) return false;

    move_history_.push(QPoint(row, col));
    emit piecePlaced(row, col, current_player_);

    if (board_.hasFiveAt(col, row, current_player_)) {
        game_over_ = true;
        emit gameOver(current_player_);
        return true;
    }

    switchPlayer();
    return true;
}
```

undo 流程与实现：

1. 弹出栈顶记录。
2. 调用 board_.forceClear(col, row) 清除该位置（forceClear 是专门为悔棋添加的方法，不检查是否有棋子）。
3. 清除 gameOver_ 标志。
4. 发射 pieceRemoved 信号。
5. 切换玩家。

```
bool Game::undo() {
    if (move_history_.empty()) return false;

    QPoint last = move_history_.top();
    move_history_.pop();
    int row = last.x(), col = last.y();

    board_.forceClear(col, row);
    game_over_ = false;

    emit pieceRemoved(row, col);

    switchPlayer();
    return true;
}
```

辅助方法：

```
void Game::reset() {
    board_.clear();
    while (!move_history_.empty()) move_history_.pop();
    current_player_ = Chess::kBlack;
    game_over_ = false;

    emit gameReset();
}
```

```

        emit currentPlayerChanged(current_player_);
    }

void Game::switchPlayer() {
    current_player_ =
        (current_player_ == Chess::kBlack) ? Chess::kWhite : Chess::kBlack;
    emit currentPlayerChanged(current_player_);
}

```

4.5. MainWindow —— 信号槽串联

MainWindow 是程序的“胶水层”，通过 connect 函数将 ChessBoardWidget 的交互事件、Game 的信号和 UI 控件连接起来。

构造函数： 初始化 Game 对象并将 Board 指针绑定到 ChessBoardWidget。

```

MainWindow::MainWindow(QWidget* parent)
    : QMainWindow(parent), ui(new Ui::MainWindow) {
    ui->setupUi(this);

    game_ = new Game(15, this);
    ui->chessBoard->set_board(&game_->board()); // 绑定数据源

    connectSignals();
}

```

信号槽连接代码：

```

void MainWindow::connectSignals() {
    // 棋盘点击 → 落子
    connect(ui->chessBoard, &ChessBoardWidget::cellClicked,
            game_, &Game::placePiece);

    // 落子 / 悔棋 → 棋盘重绘
    connect(game_, &Game::piecePlaced, ui->chessBoard,
            [this](int, int, Chess) { ui->chessBoard->update(); });
    connect(game_, &Game::pieceRemoved, ui->chessBoard,
            [this](int, int) { ui->chessBoard->update(); });

    // 游戏重置 → 棋盘重绘 + 状态栏
    connect(game_, &Game::gameReset, ui->chessBoard, [this]() {
        ui->chessBoard->update();
        statusBar()->showMessage(QStringLiteral("当前: 黑方落子"));
    });

    // 游戏结束 → 弹窗
    connect(game_, &Game::gameOver, this, [this](Chess winner) {
        QString msg = (winner == Chess::kBlack) ? QStringLiteral("黑方获胜! ")
            : QStringLiteral("白方获胜! ");
        QMessageBox::information(this, QStringLiteral("游戏结束"), msg);
    });

    // 切换玩家 → 状态栏文字
    connect(game_, &Game::currentPlayerChanged, this, [this](Chess player) {
        QString turn = (player == Chess::kBlack) ? QStringLiteral("黑方")
            : QStringLiteral("白方");
        statusBar()->showMessage(QStringLiteral("当前: ") + turn +
            QStringLiteral("落子"));
    });

    // 按钮事件
    connect(ui->newGameButton, &QPushButton::clicked, game_, &Game::reset);
    connect(ui->undoButton, &QPushButton::clicked, game_, &Game::undo);
    connect(ui->exitButton, &QPushButton::clicked, this, &MainWindow::close);
}

```


三种 connect 写法在本项目中均有使用：

- 1. **直接连接**：connect(A, &A::signal, B, &B::slot)，适合信号和槽签名完全匹配的情况（如棋盘点击 → 落子、按钮 → Game 方法）。
- 2. **Lambda 连接**：当槽需要额外逻辑（如判断颜色后构造不同字符串）时使用 lambda 表达式。
- 3. **忽略参数**：lambda 中未使用的参数只写类型不写变量名（如 (int, int, Chess)）。

4.6. 主入口与编译

main.cpp 精简为标准的 Qt 启动模板：

```
int main(int argc, char** argv) {
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

a.exec() 进入 Qt 事件循环，持续监听用户的鼠标点击、按钮操作，将事件分发给对应的对象处理。

5. 项目文件结构

```
five-in-a-row/
├── board.h           # Board 类声明（棋子枚举 + 尺寸信息）
├── board.cpp         # Board 类实现（落子、五连判定、控制台打印）
├── game.h           # Game 类声明（信号 + 接口）
├── game.cpp         # Game 类实现（落子流程、悔棋、切换玩家）
├── chessboardwidget.h # 棋盘控件声明（绘制 + 点击事件）
├── chessboardwidget.cpp # 棋盘控件实现（QPainter 绘图、坐标转换）
├── mainwindow.h     # 主窗口声明
├── mainwindow.cpp   # 主窗口实现（connect 信号槽串联）
├── mainwindow.ui     # Qt Designer UI 布局文件
├── main.cpp         # 程序入口
└── CMakeLists.txt   # CMake 构建配置
```

6. 遇到的关键问题与解决方案

此前没有 Qt 开发经验，在从纯 C++ 控制台程序迁移到 Qt 图形界面程序的过程中遇到了不少基础概念层面的困难。

6.1. 不知道如何在 Qt 中绘制图形

在控制台程序中，用 std::cout 打印字符即可在终端看到棋盘。转到 GUI 后面临的问题是：用什么工具把棋子和棋盘格子画到屏幕上？

解决过程：学习了部分视频课程，查阅 Qt 文档和示例代码后，了解到 Qt 使用 QPainter 作为绘图引擎，所有图形绘制必须写在 QWidget 的 paintEvent 虚函数中。两种方式的对比如下：

概念	控制台做法	Qt 做法
画线	无法画	painter.drawLine(x1, y1, x2, y2)
画圆（棋子）	打印 ● / ○	painter.drawEllipse(center, rx, ry)

“刷新画面”	<code>std::cout << ...</code>	<code>update()</code> 通知 Qt, Qt 自动调用 <code>paintEvent</code>
颜色控制	无	<code>setPen(Qt::black)</code> + <code>setBrush(Qt::black)</code>

理解 QPainter 的 Pen（线条属性）与 Brush（填充属性）模型后，绘制棋盘就是逐层叠加：先填背景色、再画横竖网格线、接着画星位小圆点、最后遍历 Board 数据画出所有棋子。

6.2. 不知道信号与槽（Signal & Slot）是什么、怎么使用

在控制台程序中，函数调用直接写 `game.placePiece(3, 5)` 即可。但在 GUI 程序中，用户点击棋盘发生在 `ChessBoardWidget` 内部，而落子逻辑在 `Game` 类中——两个对象互不知道对方存在，如何通信？

解决过程： 通过学习 Qt 文档，理解了信号槽机制。

第一步，在 `ChessBoardWidget` 中声明信号：

```
signals:
    void cellClicked(int row, int col);
```

第二步，在用户点击时发射信号：

```
void ChessBoardWidget::mouseReleaseEvent(QMouseEvent* event) {
    // ... 坐标转换 ...
    emit cellClicked(row, col); // 发射信号
}
```

第三步，在 `MainWindow` 中将信号和槽“接线”：

```
connect(ui->chessBoard, &ChessBoardWidget::cellClicked, // 信号发送者 + 信号名
        game_,          &Game::placePiece);           // 接收者 + 槽函数名
```

几点认识：

- 信号只需要声明，不需要实现，Qt 自动生成实现代码。
- `connect` 的前两个参数是“谁发信号”，后两个参数是“谁接收、调用什么函数”。
- 信号参数和槽参数必须类型匹配（槽可以少接收参数，多余的被忽略）。
- 声明了信号槽的类必须加上 `Q_OBJECT` 宏并继承 `QObject`。

6.3. 不知道选用什么 Qt 控件、如何搭建 UI

面对 Qt 提供的大量控件类（`QWidget`、`QFrame`、`QGraphicsView`、`QLabel` 等），不清楚应该继承哪个来实现棋盘绘制。

解决过程：

- **棋盘控件：** 直接继承 `QWidget`，重写 `paintEvent` 和 `mouseReleaseEvent`。这是最简单的方式，无需额外的布局管理，棋盘就是一块自定义的矩形区域。
- **窗口布局：** 使用 Qt Designer 的拖拽方式搭建 `mainwindow.ui` —— 左侧放置自定义的 `ChessBoardWidget`，右侧竖直排列三个 `QPushButton`。UI 文件由 CMake 中的 `qt_add_executable` 自动编译，生成的 `ui_mainwindow.h` 包含所有控件的成员变量指针。
- **按钮事件：** 通过 `connect(button, &QPushButton::clicked, receiver, &Slot)` 连接点击事件，无需手动写事件处理函数。
- **状态栏：** `QMainWindow` 自带 `statusBar()`，调用 `statusBar()->showMessage(...)` 即可显示当前玩家信息。
- **弹窗：** 使用 `QMessageBox::information(this, "标题", "内容")` 显示胜负结果。

6.4. 像素坐标与棋盘逻辑坐标的转换

棋盘绘制需要把“第 3 行第 5 列”这样的逻辑坐标映射到屏幕上的像素位置。初次实现时对公式推导不清晰。

解决过程：首先确定棋盘 $\rightarrow$ 像素的映射： $px = margin + col * cellSize$ 。这是正向映射，在 `cellToPixel` 中实现。

对于逆向映射（像素 $\rightarrow$ 棋盘），考虑到棋盘的格子是均匀排列的，采用 $px / cellSize$ 取整数商的方式来获取列号。虽然是一种近似方法，但由于 $cellSize = width / boardSize$ ，棋盘被均匀划分为 15 段，点击位置落在某个区间内即在对应的交叉点附近。同时通过 `inBound(px, py)` 先做一次范围检查，排除掉在 `Widget` 之外的无效点击。

实现代码：

```
QPoint ChessBoardWidget::cellToPixel(int row, int col) const {
    return QPoint{margin() + col * cellSize(),
                  margin() + row * cellSize()};
}

QPoint ChessBoardWidget::pixelToCell(int px, int py) const {
    int col = -1, row = -1;
    if (inBound(px, py)) {
        col = px / cellSize();
        row = py / cellSize();
    }
    return QPoint{row, col};
}
```

6.5. ChessBoardWidget 与 Board 的数据共享方式

棋盘数据存在 `Game` 的 `Board board_` 成员中，但 `ChessBoardWidget` 在 `paintEvent` 里需要读取这些数据来画棋子。如何在两个类之间共享数据？

解决过程：

- `ChessBoardWidget` 持有 `const Board*` 指针，指向 `Game` 内部的 `Board`。`paintEvent` 直接通过指针读取数据。缺点无，只需确保 `Board` 生命周期覆盖 `Widget`。

`MainWindow` 构造时调用 `ui->chessBoard->set_board(&game_>board())` 完成绑定。此后 `Widget` 只读不写（`const` 指针保证），`Board` 的修改由 `Game` 统一管理。

7. 总结与收获

本次实验从零构建了一个完整的五子棋桌面应用程序，涵盖了需求分析、架构设计、编码实现、调试修复的完整流程。

主要收获：

1. **分层架构的价值：**将数据、逻辑、视图分离后，每一层可以独立开发和测试。`Board` 层完成后通过控制台直接验证，不必等到 `GUI` 完成。这降低了调试难度。
2. **Qt 机制的掌握：**实践中理解了 `QPainter` 绘图模型、`paintEvent/update()` 的重绘机制、信号槽的声明与连接（`connect` 的三种写法）以及 `Q_OBJECT` 宏和 `MOC` 的作用。
3. **坐标转换的数学严谨性：**看似简单的 $(px - margin) / cellSize$ ，涉及整数除法、四舍五入、边距对称性等多个细节，一个符号出错就会导致棋子错位或点击无效。
4. **信号单向流动的设计：**`Game` 不知道 `UI` 的存在，`UI` 不直接操作 `Game` 的内部状态，所有交互通过 `MainWindow` 中转。这种松耦合使得组件可单独替换和测试。